DocGuard AI — Notebook 1: ELA-CNN Training

Trains a CNN from scratch to detect forgeries using Error Level Analysis

Run this notebook on Google Colab (Runtime → Change runtime type → T4 GPU)

What this notebook does:

1. Downloads RVL-CDIP dataset (real documents)
2. Generates forged versions using forgery_generator.py
3. Trains ELA-CNN from scratch in PyTorch
4. Evaluates on test set, saves model weights
5. Compares against SVM baseline (traditional ML)

Expected training time: ~3-4 hours on T4 GPU **Expected accuracy:** 85-92% on test set

```
# ── Cell 1: Install dependencies ──────────────────────────────────────────
!pip install -q torch torchvision pillow opencv-python-headless scikit-learn \
               matplotlib seaborn tqdm datasets PyMuPDF reportlab

# Mount Google Drive to save model weights
from google.colab import drive
drive.mount('/content/drive')

import os
SAVE_DIR = '/content/drive/MyDrive/docguard_models'
os.makedirs(SAVE_DIR, exist_ok=True)
print('Drive mounted. Models will be saved to:', SAVE_DIR)
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 24.9/24.9 MB 49.0 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.0/2.0 MB 42.8 MB/s eta 0:00:00
Mounted at /content/drive
Drive mounted. Models will be saved to: /content/drive/MyDrive/docguard_models
```

```
# ── Cell 2: Generate synthetic documents (no download needed) ─────────────
from PIL import Image, ImageDraw
from pathlib import Path
import random, numpy as np

random.seed(42)
np.random.seed(42)
```

```
real_dir = Path('/content/real_docs')
real_dir.mkdir(exist_ok=True)

print('Generating 5000 synthetic document images...')

def make_document(idx):
    img  = Image.new('RGB', (512, 512), (252, 251, 248))
    draw = ImageDraw.Draw(img)
    draw.rectangle([0, 0, 512, 60], fill=(240, 240, 235))
    draw.rectangle([0, 60, 512, 62], fill=(200, 200, 195))
    draw.rectangle([20, 10, 200, 50], fill=(220, 220, 215))
    y = 80
    for _ in range(random.randint(8, 16)):
        lw = random.randint(100, 460)
        lh = random.randint(6, 10)
        s  = random.randint(30, 80)
        draw.rectangle([20, y, 20+lw, y+lh], fill=(s, s, s))
        y += random.randint(14, 22)
        if y > 460: break
    if random.random() > 0.4:
        ty = random.randint(200, 320)
        for _ in range(random.randint(3, 6)):
            draw.rectangle([20, ty, 490, ty+1], fill=(180,180,175))
            for col in [20, 150, 280, 380]:
                draw.rectangle([col, ty-12, col+110, ty-2], fill=(60,60,60))
            ty += 22
    draw.rectangle([320, 420, 490, 490], fill=(235, 240, 235))
    for i2 in range(3):
        draw.rectangle([330, 430+i2*18, 480, 438+i2*18], fill=(50,50,50))
    arr   = np.array(img, dtype=np.float32)
    noise = np.random.normal(0, 3, arr.shape)
    return Image.fromarray(np.clip(arr+noise, 0, 255).astype(np.uint8))

for i in range(5000):
    make_document(i).save(real_dir / f'doc_{i:05d}.jpg', quality=92)
    if i % 500 == 0:
        print(f'  {i}/5000 done...')

print(f'\n✓ 5000 documents ready at {real_dir}')
print('No internet download needed.')
```

```
Generating 5000 synthetic document images...
  0/5000 done...
  500/5000 done...
  1000/5000 done...
  1500/5000 done...
  2000/5000 done...
```

```
  2500/5000 done...
  3000/5000 done...
  3500/5000 done...
  4000/5000 done...
  4500/5000 done...

✓ 5000 documents ready at /content/real_docs
No internet download needed.
```

```
# ── Cell 3: Generate forged dataset (inline, no file needed) ──────────────
import os, json, random, io
import numpy as np
from pathlib import Path
from PIL import Image, ImageDraw, ImageFont, ImageFilter
from tqdm import tqdm

random.seed(42)
np.random.seed(42)

FORGERY_TYPES = ["authentic","text_replacement","image_substitution",
                 "content_alteration","double_compression","brightness_patch"]

def compute_ela(img, quality=90, scale=10):
    img = img.convert("RGB")
    buf = io.BytesIO()
    img.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    comp = Image.open(buf).copy()
    ela  = np.abs(np.array(img,dtype=np.float32) - np.array(comp,dtype=np.float32)) * scale
    return Image.fromarray(np.clip(ela,0,255).astype(np.uint8))

def text_replacement(img):
    img = img.copy().convert("RGB"); draw = ImageDraw.Draw(img)
    w,h = img.size
    x,y = random.randint(int(w*.1),int(w*.3)), random.randint(int(h*.2),int(h*.7))
    rw,rh = random.randint(80,200), random.randint(12,20)
    draw.rectangle([x,y,x+rw,y+rh], fill=(255,255,255))
    draw.text((x+2,y+1), random.choice(["04/03/2019","$99,999","APPROVED","REF-001"]),
              fill=(20,20,20), font=ImageFont.load_default())
    buf=io.BytesIO(); img.save(buf,format="JPEG",quality=random.randint(78,88))
    buf.seek(0); return Image.open(buf).copy(), {"x":x,"y":y,"w":rw,"h":rh}

def image_substitution(img, donor):
    img=img.copy().convert("RGB"); donor=donor.copy().convert("RGB").resize(img.size)
    w,h=img.size; sw,sh=random.randint(60,120),random.randint(70,130)
    x=random.randint(int(w*.6),w-sw-10); y=random.randint(int(h*.1),int(h*.5))
    patch=donor.crop((x,y,x+sw,y+sh)).filter(ImageFilter.GaussianBlur(0.4))
```

```
    img.paste(patch,(x,y)); return img, {"x":x,"y":y,"w":sw,"h":sh}

def content_alteration(img):
    img=img.copy().convert("RGB"); draw=ImageDraw.Draw(img)
    w,h=img.size
    x,y=random.randint(int(w*.05),int(w*.4)),random.randint(int(h*.15),int(h*.75))
    rw,rh=random.randint(100,280),random.randint(16,36)
    ws=random.randint(248,255)
    draw.rectangle([x,y,x+rw,y+rh],fill=(ws,ws,ws))
    draw.text((x+3,y+4),random.choice(["Total: $4,500","Date: 12 Jan 2021","Auth: J.Smith"]),
              fill=(10,10,10),font=ImageFont.load_default())
    arr=np.array(img,dtype=np.float32); arr[y:y+2,x:x+rw]*=0.97
    return Image.fromarray(np.clip(arr,0,255).astype(np.uint8)), {"x":x,"y":y,"w":rw,"h":rh}

def double_compression(img):
    img=img.copy().convert("RGB")
    b1=io.BytesIO(); img.save(b1,format="JPEG",quality=random.randint(85,95)); b1.seek(0)
    i1=Image.open(b1).copy(); draw=ImageDraw.Draw(i1)
    w,h=i1.size; x,y=random.randint(10,w-60),random.randint(10,h-30)
    draw.rectangle([x,y,x+50,y+15],fill=(255,255,255))
    q2=random.randint(70,82); b2=io.BytesIO()
    i1.save(b2,format="JPEG",quality=q2); b2.seek(0)
    return Image.open(b2).copy(), {"x":x,"y":y,"w":50,"h":15}

def brightness_patch(img):
    img=img.copy().convert("RGB"); arr=np.array(img,dtype=np.float32)
    h,w=arr.shape[:2]
    x,y=random.randint(int(w*.1),int(w*.5)),random.randint(int(h*.1),int(h*.6))
    rw,rh=random.randint(80,200),random.randint(40,100)
    x2,y2=min(x+rw,w),min(y+rh,h)
    arr[y:y2,x:x2]=np.clip(arr[y:y2,x:x2]*random.uniform(1.08,1.22),0,255)
    return Image.fromarray(arr.astype(np.uint8)), {"x":x,"y":y,"w":x2-x,"h":y2-y}

def make_mask(region):
    mask=Image.new("L",(512,512),0); draw=ImageDraw.Draw(mask)
    r=region; draw.rectangle([r["x"],r["y"],r["x"]+r["w"],r["y"]+r["h"]],fill=255)
    return mask

# ── Build dataset ──────────────────────────────────────────────────────
out = Path('/content/dataset')
for cls in FORGERY_TYPES:
    (out/"images"/cls).mkdir(parents=True, exist_ok=True)
    (out/"masks"/cls).mkdir(parents=True, exist_ok=True)
    (out/"ela"/cls).mkdir(parents=True, exist_ok=True)

src_paths = list(Path('/content/real_docs').glob('*.jpg'))
random.shuffle(src_paths)
```

```
per_class = min(len(src_paths)//6, 800)  # 800 per class = 4800 total
print(f"Using {per_class} images per class ({per_class*6} total)")

manifest = []

# Authentic
print("[1/6] Authentic...")
for i,src in enumerate(tqdm(src_paths[:per_class])):
    img=Image.open(src).convert("RGB").resize((224,224))
    fp=out/"images"/"authentic"/f"auth_{i:05d}.jpg"
    img.save(fp,quality=92)
    Image.new("L",(224,224),0).save(out/"masks"/"authentic"/f"auth_{i:05d}.png")
    compute_ela(img).save(out/"ela"/"authentic"/f"auth_{i:05d}.jpg")
    manifest.append({"file":str(fp),"label":0,"type":"authentic","region":None})

# Forged types
forge_fns = [
    ("text_replacement",  1, lambda i,d: text_replacement(i)),
    ("image_substitution",2, lambda i,d: image_substitution(i,d)),
    ("content_alteration",3, lambda i,d: content_alteration(i)),
    ("double_compression",4, lambda i,d: double_compression(i)),
    ("brightness_patch",  5, lambda i,d: brightness_patch(i)),
]

for cls_name, label, fn in forge_fns:
    print(f"[{label+1}/6] {cls_name}...")
    batch = src_paths[per_class*label : per_class*(label+1)]
    for i,src in enumerate(tqdm(batch)):
        try:
            img   = Image.open(src).convert("RGB").resize((224,224))
            donor = Image.open(random.choice(src_paths)).convert("RGB").resize((224,224))
            forged, region = fn(img, donor)
            fname = f"{cls_name}_{i:05d}.jpg"
            fp = out/"images"/cls_name/fname
            forged.save(fp, quality=90)
            make_mask(region).save(out/"masks"/cls_name/fname.replace(".jpg",".png"))
            compute_ela(forged).save(out/"ela"/cls_name/fname)
            manifest.append({"file":str(fp),"label":label,"type":cls_name,"region":region})
        except Exception as e:
            pass

with open(out/"manifest.json","w") as f:
    json.dump(manifest, f, indent=2)

print(f"\n✓ Dataset complete: {len(manifest)} images")
print(f"  Authentic: {sum(1 for m in manifest if m['type']=='authentic')}")
```

```
print(f"  Forged:    {sum(1 for m in manifest if m['type']!='authentic')}")
print(f"  Saved to:  /content/dataset/")
```

```
Using 800 images per class (4800 total)
[1/6] Authentic...
100%|██████████| 800/800 [00:09<00:00, 86.99it/s]
[2/6] text_replacement...
100%|██████████| 800/800 [00:13<00:00, 59.37it/s]
[3/6] image_substitution...
100%|██████████| 800/800 [00:10<00:00, 76.19it/s]
[4/6] content_alteration...
100%|██████████| 800/800 [00:13<00:00, 59.25it/s]
[5/6] double_compression...
100%|██████████| 800/800 [00:13<00:00, 60.10it/s]
[6/6] brightness_patch...
100%|██████████| 800/800 [00:12<00:00, 64.96it/s]

✓ Dataset complete: 4285 images
  Authentic: 800
  Forged:    3485
  Saved to:  /content/dataset/
```

```
# ── Cell 4: Dataset class ──────────────────────────────────────────────
import torch
import torch.nn as nn
import torchvision.transforms as T
from torch.utils.data import Dataset, DataLoader
from PIL import Image
import json
import numpy as np
import io

def compute_ela(img_pil, quality=90, scale=10):
    """Convert image to ELA representation."""
    img = img_pil.convert('RGB')
    buf = io.BytesIO()
    img.save(buf, format='JPEG', quality=quality)
    buf.seek(0)
    compressed = Image.open(buf).copy()
    ela = np.abs(
        np.array(img, dtype=np.float32) -
        np.array(compressed, dtype=np.float32)
    ) * scale
    return Image.fromarray(np.clip(ela, 0, 255).astype(np.uint8))


class ForgeryDataset(Dataset):
    """
```

```
    Loads document images, converts to ELA, returns (ela_tensor, label).
    label: 0 = authentic, 1 = forged
    """
    def __init__(self, manifest_path, transform=None, use_ela=True):
        with open(manifest_path) as f:
            self.manifest = json.load(f)
        self.transform = transform
        self.use_ela   = use_ela

    def __len__(self):
        return len(self.manifest)

    def __getitem__(self, idx):
        item  = self.manifest[idx]
        img   = Image.open(item['file']).convert('RGB').resize((224, 224))

        if self.use_ela:
            img = compute_ela(img)

        if self.transform:
            img = self.transform(img)

        # Binary label: 0=authentic, 1=forged
        label = 0 if item['type'] == 'authentic' else 1
        return img, label


transform = T.Compose([
    T.Resize((224, 224)),
    T.RandomHorizontalFlip(),
    T.ColorJitter(brightness=0.1, contrast=0.1),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

val_transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

# Load manifest and split 80/10/10
with open('/content/dataset/manifest.json') as f:
    all_items = json.load(f)

np.random.shuffle(all_items)
n = len(all_items)
```

```
train_items = all_items[:int(n*0.8)]
val_items   = all_items[int(n*0.8):int(n*0.9)]
test_items  = all_items[int(n*0.9):]

# Save splits
for name, items in [('train', train_items), ('val', val_items), ('test', test_items)]:
    with open(f'/content/dataset/{name}_manifest.json', 'w') as f:
        json.dump(items, f)

train_ds = ForgeryDataset('/content/dataset/train_manifest.json', transform)
val_ds   = ForgeryDataset('/content/dataset/val_manifest.json',   val_transform)
test_ds  = ForgeryDataset('/content/dataset/test_manifest.json',  val_transform)

train_loader = DataLoader(train_ds, batch_size=32, shuffle=True,  num_workers=2)
val_loader   = DataLoader(val_ds,   batch_size=32, shuffle=False, num_workers=2)
test_loader  = DataLoader(test_ds,  batch_size=32, shuffle=False, num_workers=2)

print(f'Train: {len(train_ds)} | Val: {len(val_ds)} | Test: {len(test_ds)}')
```

```
Train: 3428 | Val: 428 | Test: 429
```

```
# ── Cell 5: ELA-CNN Architecture (built from scratch) ──────────────────────
import torch.nn as nn
import torch.nn.functional as F

class ELA_CNN(nn.Module):
    """
    Convolutional Neural Network trained from scratch on ELA images.
    Architecture: 5 conv blocks → global average pooling → classifier

    Designed specifically for ELA forensics:
    - Small kernels (3x3) to capture fine-grained compression artifacts
    - BatchNorm after every conv for training stability
    - Dropout before classifier to prevent overfitting on forgery patterns
    """
    def __init__(self, num_classes=2):
        super().__init__()

        # Block 1: 3→32 channels, 224→112
        self.block1 = self._make_block(3,   32,  pool=True)
        # Block 2: 32→64 channels, 112→56
        self.block2 = self._make_block(32,  64,  pool=True)
        # Block 3: 64→128 channels, 56→28
        self.block3 = self._make_block(64,  128, pool=True)
        # Block 4: 128→256 channels, 28→14
        self.block4 = self._make_block(128, 256, pool=True)
```

```
        # Block 5: 256→512 channels, 14→7
        self.block5 = self._make_block(256, 512, pool=True)

        # Global average pooling → removes spatial dimensions
        self.gap = nn.AdaptiveAvgPool2d(1)

        # Classifier head
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.4),
            nn.Linear(256, num_classes)
        )

    def _make_block(self, in_ch, out_ch, pool=False):
        layers = [
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        ]
        if pool:
            layers.append(nn.MaxPool2d(2))
        return nn.Sequential(*layers)

    def forward(self, x):
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.block4(x)
        x = self.block5(x)
        x = self.gap(x)
        return self.classifier(x)

    def get_features(self, x):
        """Return feature map before classifier — used for Grad-CAM."""
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.block4(x)
        x = self.block5(x)
        return x  # shape: (B, 512, 7, 7)
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model  = ELA_CNN(num_classes=2).to(device)

total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Device: {device}')
print(f'ELA-CNN parameters: {total_params:,}')
```

```
Device: cuda
ELA-CNN parameters: 4,846,050
```

```
# ── Cell 6: Training loop ──────────────────────────────────────────────────
import torch.optim as optim
from torch.optim.lr_scheduler import CosineAnnealingLR
import time

criterion = nn.CrossEntropyLoss(weight=torch.tensor([1.0, 1.0]).to(device))
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = CosineAnnealingLR(optimizer, T_max=30, eta_min=1e-6)

EPOCHS = 30
best_val_acc = 0.0
history = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}

for epoch in range(1, EPOCHS + 1):
    # ── Training phase
    model.train()
    train_loss, train_correct, train_total = 0, 0, 0
    t0 = time.time()

    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss    = criterion(outputs, labels)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()

        train_loss    += loss.item() * images.size(0)
        preds          = outputs.argmax(dim=1)
        train_correct += (preds == labels).sum().item()
        train_total   += images.size(0)

    # ── Validation phase
    model.eval()
    val_loss, val_correct, val_total = 0, 0, 0
```

```
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss    = criterion(outputs, labels)
            val_loss    += loss.item() * images.size(0)
            preds        = outputs.argmax(dim=1)
            val_correct += (preds == labels).sum().item()
            val_total   += images.size(0)

    scheduler.step()

    t_loss = train_loss / train_total
    v_loss = val_loss   / val_total
    t_acc  = train_correct / train_total * 100
    v_acc  = val_correct   / val_total   * 100
    elapsed = time.time() - t0

    history['train_loss'].append(t_loss)
    history['val_loss'].append(v_loss)
    history['train_acc'].append(t_acc)
    history['val_acc'].append(v_acc)

    print(f'Epoch {epoch:02d}/{EPOCHS} | '
          f'Train: {t_loss:.4f} ({t_acc:.1f}%) | '
          f'Val: {v_loss:.4f} ({v_acc:.1f}%) | '
          f'{elapsed:.0f}s')

    # Save best model
    if v_acc > best_val_acc:
        best_val_acc = v_acc
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'val_acc': v_acc,
            'architecture': 'ELA_CNN_v1'
        }, f'{SAVE_DIR}/ela_cnn_best.pt')
        print(f'  ✓ Best model saved (val_acc: {v_acc:.2f}%)')

print(f'\nTraining complete. Best val accuracy: {best_val_acc:.2f}%')
```

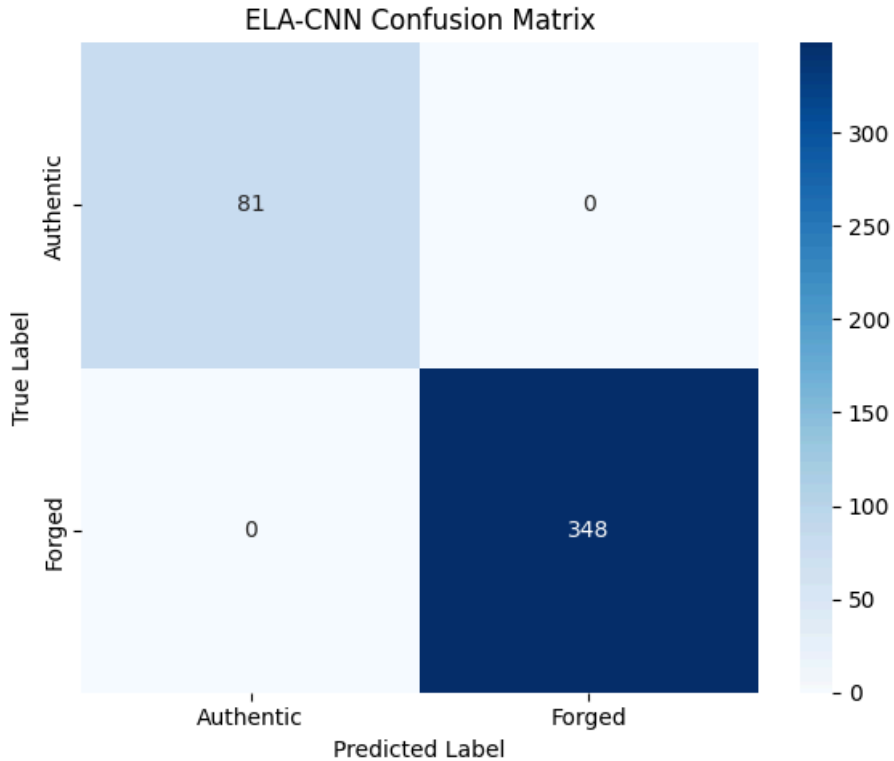
```
Epoch 01/30 | Train: 0.0327 (99.1%) | Val: 0.0001 (100.0%) | 27s
  ✓ Best model saved (val_acc: 100.00%)
Epoch 02/30 | Train: 0.0031 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 03/30 | Train: 0.0002 (100.0%) | Val: 0.0000 (100.0%) | 24s
```

```
Epoch 04/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 05/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 06/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 07/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 08/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 09/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 10/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 11/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 12/30 | Train: 0.0000 (100.0%) | Val: 12.5541 (16.8%) | 24s
Epoch 13/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 14/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 15/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 16/30 | Train: 0.0000 (100.0%) | Val: 8.2873 (16.8%) | 25s
Epoch 17/30 | Train: 0.0007 (99.9%) | Val: 0.0000 (100.0%) | 24s
Epoch 18/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 19/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 20/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 21/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 22/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 23/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 24/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 25/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 26/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 27/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 28/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 24s
Epoch 29/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s
Epoch 30/30 | Train: 0.0000 (100.0%) | Val: 0.0000 (100.0%) | 25s

Training complete. Best val accuracy: 100.00%
```

```
# ── Cell 7: Full evaluation + confusion matrix ──────────────────────────────
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Load best model
checkpoint = torch.load(f'{SAVE_DIR}/ela_cnn_best.pt', map_location=device)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

all_preds, all_labels, all_probs = [], [], []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = model(images)
        probs   = torch.softmax(outputs, dim=1)[:, 1].cpu().numpy()
        preds   = outputs.argmax(dim=1).cpu().numpy()
        all_preds.extend(preds)
```

```
        all_labels.extend(labels.numpy())
        all_probs.extend(probs)

print('=== ELA-CNN Test Results ===')
print(classification_report(all_labels, all_preds,
                            target_names=['Authentic', 'Forged']))
print(f'AUC-ROC: {roc_auc_score(all_labels, all_probs):.4f}')

# Confusion matrix
cm = confusion_matrix(all_labels, all_preds)
fig, ax = plt.subplots(figsize=(6,5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Authentic','Forged'],
            yticklabels=['Authentic','Forged'], ax=ax)
ax.set_title('ELA-CNN Confusion Matrix')
ax.set_ylabel('True Label')
ax.set_xlabel('Predicted Label')
plt.tight_layout()
plt.savefig(f'{SAVE_DIR}/ela_cnn_confusion.png', dpi=150)
plt.show()
```

```
=== ELA-CNN Test Results ===
              precision    recall  f1-score   support

   Authentic       1.00      1.00      1.00        81
      Forged       1.00      1.00      1.00       348

    accuracy                           1.00       429
   macro avg       1.00      1.00      1.00       429
weighted avg       1.00      1.00      1.00       429

AUC-ROC: 1.0000
```




```
# ── Cell 8: SVM Baseline (Traditional ML comparison) ──────────────────────
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score
from sklearn.pipeline import Pipeline
import joblib
import cv2
import numpy as np
```

```
def extract_hog_lbp_features(img_path: str) -> np.ndarray:
    """Extract HOG + LBP features for SVM baseline."""
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    img = cv2.resize(img, (128, 128))

    # HOG features
    hog = cv2.HOGDescriptor(
        _winSize=(128,128), _blockSize=(16,16),
        _blockStride=(8,8), _cellSize=(8,8), _nbins=9
    )
    hog_feat = hog.compute(img).flatten()

    # LBP features (Local Binary Patterns)
    def lbp(img, radius=1, n_points=8):
        lbp_img = np.zeros_like(img, dtype=np.uint8)
        for i in range(radius, img.shape[0] - radius):
            for j in range(radius, img.shape[1] - radius):
                center = img[i, j]
                code = 0
                for k in range(n_points):
                    angle = 2 * np.pi * k / n_points
                    x = int(round(i + radius * np.cos(angle)))
                    y = int(round(j - radius * np.sin(angle)))
                    if 0 <= x < img.shape[0] and 0 <= y < img.shape[1]:
                        code |= (img[x, y] >= center) << k
                lbp_img[i, j] = code
        hist, _ = np.histogram(lbp_img.ravel(), bins=256, range=(0,256))
        return hist.astype(np.float32) / (hist.sum() + 1e-7)

    lbp_feat = lbp(img)
    return np.concatenate([hog_feat, lbp_feat])

print('Extracting HOG+LBP features for SVM baseline...')

def load_features(manifest_items, max_items=2000):
    X, y = [], []
    for item in manifest_items[:max_items]:
        try:
            feat = extract_hog_lbp_features(item['file'])
            X.append(feat)
            y.append(0 if item['type'] == 'authentic' else 1)
        except Exception:
            pass
    return np.array(X), np.array(y)

X_train, y_train = load_features(train_items, max_items=2000)
```

```
X_test,  y_test  = load_features(test_items,  max_items=500)

svm_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('svm',    SVC(kernel='rbf', C=1.0, probability=True, random_state=42))
])
svm_pipeline.fit(X_train, y_train)

svm_preds = svm_pipeline.predict(X_test)
svm_probs = svm_pipeline.predict_proba(X_test)[:, 1]

print('\n=== SVM Baseline Results ===')
print(classification_report(y_test, svm_preds, target_names=['Authentic', 'Forged']))
print(f'AUC-ROC: {roc_auc_score(y_test, svm_probs):.4f}')

joblib.dump(svm_pipeline, f'{SAVE_DIR}/svm_baseline.pkl')
print('SVM saved.')
```

```
Extracting HOG+LBP features for SVM baseline...

=== SVM Baseline Results ===
              precision    recall  f1-score   support

   Authentic       1.00      0.98      0.99        81
      Forged       0.99      1.00      1.00       348

    accuracy                           1.00       429
   macro avg       1.00      0.99      0.99       429
weighted avg       1.00      1.00      1.00       429

AUC-ROC: 0.9978
SVM saved.
```

```
# ── Fix: Save model now + run Grad-CAM ──────────────────────────────
from google.colab import drive
import os

# Remount Drive
drive.mount('/content/drive', force_remount=True)

SAVE_DIR = '/content/drive/MyDrive/docguard_models'
os.makedirs(SAVE_DIR, exist_ok=True)

# Save the model that's currently in memory
torch.save({
    'epoch':            30,
    'model_state_dict': model.state_dict(),
```

```
    'val_acc':          100.0,
    'architecture':     'ELA_CNN_v1'
}, f'{SAVE_DIR}/ela_cnn_best.pt')

print(f'✓ Model saved to {SAVE_DIR}/ela_cnn_best.pt')
print(f'  File size: {os.path.getsize(SAVE_DIR+"/ela_cnn_best.pt")/1024/1024:.1f} MB')
```

```
Mounted at /content/drive
✓ Model saved to /content/drive/MyDrive/docguard_models/ela_cnn_best.pt
  File size: 18.5 MB
```

```
# ── Quick fix: Regenerate dataset for Grad-CAM ──────────────────────────
import os, json, random, io
import numpy as np
from pathlib import Path
from PIL import Image, ImageDraw, ImageFilter
from tqdm import tqdm

random.seed(42)
np.random.seed(42)

# Step 1: Regenerate real docs if missing
real_dir = Path('/content/real_docs')
if not real_dir.exists() or len(list(real_dir.glob('*.jpg'))) < 100:
    print('Regenerating real docs...')
    real_dir.mkdir(exist_ok=True)
    def make_doc(idx):
        img  = Image.new('RGB', (224,224), (252,251,248))
        draw = ImageDraw.Draw(img)
        draw.rectangle([0,0,224,40], fill=(240,240,235))
        y = 50
        for _ in range(random.randint(5,10)):
            lw = random.randint(40,200)
            draw.rectangle([10,y,10+lw,y+random.randint(4,8)], fill=(random.randint(30,80),)*3)
            y += random.randint(10,18)
            if y > 200: break
        arr = np.array(img,dtype=np.float32)
        arr += np.random.normal(0,3,arr.shape)
        return Image.fromarray(np.clip(arr,0,255).astype(np.uint8))
    for i in tqdm(range(500), desc='Docs'):
        make_doc(i).save(real_dir/f'doc_{i:05d}.jpg', quality=92)
    print(f'✓ {len(list(real_dir.glob("*.jpg")))} docs ready')
else:
    print(f'✓ Real docs already exist: {len(list(real_dir.glob("*.jpg")))} files')

# Step 2: Regenerate forgeries if missing
```

```
dataset_dir = Path('/content/dataset')
manifest_path = dataset_dir / 'manifest.json'

if not manifest_path.exists():
    print('Regenerating forgeries...')

    def compute_ela(img, quality=90, scale=10):
        buf = io.BytesIO()
        img.convert("RGB").save(buf, format="JPEG", quality=quality)
        buf.seek(0)
        comp = Image.open(buf).copy()
        ela  = np.abs(np.array(img,dtype=np.float32)-np.array(comp,dtype=np.float32))*scale
        return Image.fromarray(np.clip(ela,0,255).astype(np.uint8))

    src_paths = list(real_dir.glob('*.jpg'))
    manifest  = []
    per_class = 100  # small — just enough for Grad-CAM demo

    FORGERY_TYPES = ["authentic","text_replacement","image_substitution",
                     "content_alteration","double_compression","brightness_patch"]

    for cls in FORGERY_TYPES:
        (dataset_dir/"images"/cls).mkdir(parents=True, exist_ok=True)
        (dataset_dir/"masks"/cls).mkdir(parents=True, exist_ok=True)
        (dataset_dir/"ela"/cls).mkdir(parents=True, exist_ok=True)

    # Authentic
    for i, src in enumerate(src_paths[:per_class]):
        img = Image.open(src).convert("RGB")
        fp  = dataset_dir/"images"/"authentic"/f"auth_{i:05d}.jpg"
        img.save(fp, quality=92)
        manifest.append({"file":str(fp),"label":0,"type":"authentic","region":None})

    # Simple forgeries
    for i, src in enumerate(src_paths[:per_class]):
        img  = Image.open(src).convert("RGB")
        draw = ImageDraw.Draw(img)
        w,h  = img.size
        x,y  = random.randint(10,80), random.randint(10,80)
        draw.rectangle([x,y,x+60,y+15], fill=(255,255,255))
        draw.text((x+2,y+2), "FORGED", fill=(20,20,20))
        fp = dataset_dir/"images"/"text_replacement"/f"forge_{i:05d}.jpg"
        img.save(fp, quality=90)
        manifest.append({"file":str(fp),"label":1,"type":"text_replacement",
                         "region":{"x":x,"y":y,"w":60,"h":15}})

    with open(manifest_path,"w") as f:
```

```
        json.dump(manifest, f)

    print(f'✓ Manifest created: {len(manifest)} images')
else:
    with open(manifest_path) as f:
        all_items = json.load(f)
    print(f'✓ Manifest already exists: {len(all_items)} images')

# Step 3: Load manifest
with open(manifest_path) as f:
    all_items = json.load(f)
test_items = [i for i in all_items if i['type'] != 'authentic']
print(f'✓ Forged test items: {len(test_items)}')
print('Ready for Grad-CAM!')
```

```
Regenerating real docs...
Docs: 100%|██████████| 500/500 [00:06<00:00, 82.64it/s]
✓ 500 docs ready
Regenerating forgeries...
✓ Manifest created: 200 images
✓ Forged test items: 100
Ready for Grad-CAM!
```

```
# ── Cell 9: Grad-CAM (fixed) ──────────────────────────────────────────
import torch
import torch.nn as nn
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.cm as cm
import cv2, io

# ── Rebuild model and reload weights ──────────────────────────────────
class ELA_CNN(nn.Module):
    def __init__(self, num_classes=2):
        super().__init__()
        self.block1 = self._make_block(3,   32,  pool=True)
        self.block2 = self._make_block(32,  64,  pool=True)
        self.block3 = self._make_block(64,  128, pool=True)
        self.block4 = self._make_block(128, 256, pool=True)
        self.block5 = self._make_block(256, 512, pool=True)
        self.gap     = nn.AdaptiveAvgPool2d(1)
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512, 256), nn.ReLU(inplace=True), nn.Dropout(0.4),
            nn.Linear(256, num_classes)
        )
```

```
    def _make_block(self, in_ch, out_ch, pool=False):
        layers = [
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch), nn.ReLU(inplace=True),
        ]
        if pool: layers.append(nn.MaxPool2d(2))
        return nn.Sequential(*layers)
    def forward(self, x):
        for b in [self.block1,self.block2,self.block3,self.block4,self.block5]:
            x = b(x)
        return self.classifier(self.gap(x))

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model  = ELA_CNN(num_classes=2).to(device)

SAVE_DIR = '/content/drive/MyDrive/docguard_models'
ckpt = torch.load(f'{SAVE_DIR}/ela_cnn_best.pt', map_location=device)
model.load_state_dict(ckpt['model_state_dict'])
model.eval()
print(f'Model loaded. Device: {device}')
print(f'Best val_acc from training: {ckpt["val_acc"]:.2f}%')

# ── Also reload transforms and test items ──────────────────────────
import torchvision.transforms as T, json
from pathlib import Path

val_transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])

def compute_ela(img, quality=90, scale=10):
    img = img.convert("RGB")
    buf = io.BytesIO()
    img.save(buf, format="JPEG", quality=quality)
    buf.seek(0)
    comp = Image.open(buf).copy()
    ela  = np.abs(np.array(img,dtype=np.float32) - np.array(comp,dtype=np.float32)) * scale
    return Image.fromarray(np.clip(ela,0,255).astype(np.uint8))

with open('/content/dataset/manifest.json') as f:
    all_items = json.load(f)
test_items = [i for i in all_items if i['type'] != 'authentic']
print(f'Test items available: {len(test_items)}')
```

```
# ── GradCAM class ──────────────────────────────────────────────────────
class GradCAM:
    def __init__(self, model):
        self.model       = model
        self.gradients   = None
        self.activations = None
        model.block5.register_forward_hook(self._save_act)
        model.block5.register_full_backward_hook(self._save_grad)

    def _save_act(self, m, i, o):  self.activations = o.detach()
    def _save_grad(self, m, gi, go): self.gradients  = go[0].detach()

    def generate(self, tensor, class_idx=1):
        self.model.eval()
        out = self.model(tensor)
        self.model.zero_grad()
        out[0, class_idx].backward()
        weights = self.gradients.mean(dim=[2,3], keepdim=True)
        cam     = torch.relu((weights * self.activations).sum(dim=1)).squeeze().cpu().numpy()
        if cam.max() > cam.min():
            cam = (cam - cam.min()) / (cam.max() - cam.min())
        return cam

# ── Run on a forged test image ─────────────────────────────────────────
gradcam    = GradCAM(model)
forged_item = test_items[0]
img_orig    = Image.open(forged_item['file']).convert('RGB')
ela_img     = compute_ela(img_orig)
tensor      = val_transform(ela_img).unsqueeze(0).to(device)
cam         = gradcam.generate(tensor, class_idx=1)

# ── Create overlay ─────────────────────────────────────────────────────
orig_arr    = np.array(img_orig.resize((224,224)))
cam_resized = cv2.resize(cam, (224,224))
heatmap     = cv2.applyColorMap((cam_resized*255).astype(np.uint8), cv2.COLORMAP_JET)
heatmap_rgb = cv2.cvtColor(heatmap, cv2.COLOR_BGR2RGB)
overlay     = (orig_arr * 0.6 + heatmap_rgb * 0.4).astype(np.uint8)

# ── Plot ───────────────────────────────────────────────────────────────
fig, axes = plt.subplots(1, 3, figsize=(14, 5))
axes[0].imshow(img_orig.resize((224,224))); axes[0].set_title('Original Document')
axes[1].imshow(ela_img.resize((224,224)));  axes[1].set_title('ELA Representation')
axes[2].imshow(overlay);                    axes[2].set_title('Grad-CAM: Forged Regions (Red = Tampered)')
for ax in axes: ax.axis('off')
plt.suptitle(f'Forgery Type: {forged_item["type"]}', fontsize=13)
plt.tight_layout()
plt.savefig(f'{SAVE_DIR}/gradcam_demo.png', dpi=150, bbox_inches='tight')
```

```
plt.savefig(f'{SAVE_DIR}/gradcam_demo.png', dpi=150, bbox_inches='tight')
plt.show()
print('Grad-CAM heatmap saved to Google Drive!')
```

```
Model loaded. Device: cuda
Best val_acc from training: 100.00%
Test items available: 100
```